PROCESSES, SHELLS AND SCHELL SCRIPT

PROCESS COMMANDS

ps

- 'Process status'
- Utility for viewing information concerning a selection of running processes on a system
- Reads information from the virtual files in /proc filesystem

```
 PID  TTY        TIME  CMD
1361  pts/0  00:00:01  bash
1832  pts/0  00:00:00  ps
```

Additional info can be found at: https://www.tecmint.com/ps-command-examples-for-linux-process-monitoring/

PROCESS COMMANDS

kill

- A built-in command which is used to terminate processes manually
- Sends a signal to a process which terminates the process
- Normally used ad:

```
kill <pid>
```

UNIX/LINUX SHELL

- There are several shells available in Unix/Linux
  - C shell (csh)
  - Bourne Again Shell (bash)
- Different shells have different characteristics
  - Most commands and features are almost the same
- Unix systems uses the C shell or TC shell (tcsh)
- Linux systems uses bash

ALIASES

- Used to substitute a command for another
- Commonly used to map longer commands to shorter key sequences.
- When created, exists only for the shell where the alias was created and will be deleted once the system is rebooted unless added to the .bashrc file of the user home directory
- To create or an alias:

  alias [name]='[command sequence]'

  Ex: alias getpass='sudo more /etc/passwd'
- To delete an alias:

  unalias [name] Ex: unalias getpass

SETTING SHELL CHARACTERISTICS

- Shell-wide characteristics are dependent on flags that control how the shell functions
- Use –o to turn on, +o to turn off

set [-o / +o] [option name]

- Example options:
  - noclobber - prevent files from being overwritten by redirection
  - verbose – print shell input lines as they are read

BASIC SCRIPTING

- Shell scripts is a file of executable commands that has been stored in a text file.
- Commonly used to automate tasks or repetitive commands
- Can use any shell command and logic operators
- To create scripts, you'll need
  - Variables
  - Input commands
  - Operators
  - Conditional Statements
  - Loops

SHELL VARIABLES

- Feature that allows storing of data
- Variables are given names and stored temporarily in memory
- Lost when a terminal window or shell is closed unless saved in .bash_profile of the user home folder
- In a BASH Shell, variables may be
  - Local
    - Associated with user tasks and available only for the shell session where it is created.
    - Usually lower case in convention
    - To create or change a variable:

      [Variable name] = [value] Ex: price=100

      [Variable name] = `[command]` Ex: directory=`pwd`
    - To retrieve value:

      $[Variable name] Ex: echo $price

SHELL VARIABLES

- In a BASH Shell, variables may be
  - Environment
    - Used by the shell in interpreting commands and performing tasks
    - Available to current and all subshells created
    - To create or change a variable:

      export [Variable name] = [value] Ex: export HISTSIZE=100

      export [existing variable name] Ex: export $HISTSIZE
    - To retrieve value:

      $[Variable name] Ex: echo $HISTSIZE
    - To view existing environment variables:

      env

ACCEPTING INPUT

read [variablename]

- Accepts a string from the keyboard and saves it to a variable
- Example: read VAR1

SPECIAL CHARACTERS

- Parentheses ( )
  - Used to group commands together
  - Ex: (pwd; ls –l) > out.txt
- Double parentheses (())
  - Used for arithmetic operations
  - Ex: echo $((1+1)
- Braces { }
  - Used to delimit variable names
  - Useful when trying to concatenate variable values with another string
  - Ex: echo ${var1}_hello
- Backslash \
  - Escape character used so that special characters are printed instead of interpreted by the shell
  - Ex: echo \$2

CONTROL STATEMENTS

- Allow multiple commands to be run at once or to run depending on success of a previous command.
- Semicolon (;)
  - Used to run commands in consecutively and independent of each other
  - Ex: `cal 1 2016; cal 2 2016; cal3 2016`
- Double Ampersand (&&)
  - Acts like a logical 'and'
  - Used to run the next command only if the previous one was run successfully
  - Ex: `rm file1 && echo 'Success!'`
- Double Pipe (||)
  - Acts like a logical 'or'
  - Skips command 2 if command 1 is successful and vise versa
  - Ex: `rm file1 || echo 'Delete failed'`

COMPARATORS

test [val1] [operator] [val2]

- Outputs a 1 or a 0 exit code depending on results of the operation
- Operators:

Operator Type	Available Operators	Example
File	-e (exists)	`test -e file1`
String	=, !=, /<, />	`test "a" = "a"`
Arithmetic	-eq, -ne, -le, -lt, -ge, -gt	`test $var eq 1`
Logic	-a, -o	`test 1 -eq 1 -a 2 -eq 2`

- Note:
  - Command can also use `[ expression ]` instead of "test" e.g. `[ "a" = "a" ]`
  - Each value is considered a command argument – spaces are important!

BASIC SCRIPTING

- Basic script content:

Means spawn a new BASH shell

```
#!/bin/sh
# This is a comment
echo Hello World          # This is also a comment
```

- To run scripts: `./[script path/name]`
  - Make sure to set the 'x' flag using chmod

CONDITIONALS – IF/ELSE

```
if command / condition
then
     do something
Elif command / condition
then
     do something else
else
     do something else
fi
```

Example:

```
read var1
if [ $var1 -eq 0 ]
then
     echo "Zero"
Elif [ $var1 -gt 1]
then
     echo "Positive"
else
     echo "Negative"
fi
```

CONDITIONALS - CASE

```
case "variable" in
value1)
      do something
      ;;
value2)
      do something else
      ;;
*)
      do default action
esac
```

Example:

```
echo "Delete file1? (Y/N)"
read var1
case "$var1" in
Y|y)
      rm file1
      echo "file1 deleted."
      ;;
N|n)
      echo "Did not delete."
      ;;
*)
      echo "Invalid input."
esac
```

LOOPS - WHILE

```
while condition
do
    do something
done
```

Example :

```
i=1
while [ i lt 6 ]
do
     echo "printing $i"
     i=$(($i + 1))
done
```

LOOPS - FOR

```
for var in list
do
  do something
done
```

Example 1:

```
for i in 1 2 3 4 5
do
      echo $i
done
```

Example 2:

```
list="str1 str2 str3 str4"
for i in $list
do
      echo $i
done
```